



**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT A
CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY) BALMATTA,
MANGALORE**

FINAL PROJECT REPORT

ON

CLOUD COST INTELLIGENCE SYSTEM FOR HEALTHCARE

SUBMITTED BY

MOHAMMED MUNEEB

REG NO :

23BCAICD063

GUIDED BY

MISS DEEPTHI K

SUBMISSION DATE

29-04-2026

Table of Contents

1 EXECUTIVE SUMMARY

2 BACKGROUND

2.1 AIM AND OBJECTIVES

2.1.1 Aim

2.1.2 Objectives

2.2 TECHNOLOGIES USED

2.2.1 Python

2.2.2 Pandas

2.2.3 Numpy

2.2.4 Scikit-learn

2.2.5 Streamlit

2.2.6 Matplotlib

2.3 HARDWARE ARCHITECTURE

2.4 SOFTWARE ARCHITECTURE

3 SYSTEM OVERVIEW

3.1 Requirements

3.1.1 Functional Requirements

3.1.2 User Requirements

3.1.3 Environmental Requirements

3.2 Design and Architecture

3.3 Implementation

3.3.1 Data Input and Validation

3.3.2 Cost Analysis and Aggregation

3.3.3 Service-wise Cost Distribution

3.3.4 Service Usage Analysis

- 3.3.5 Department-wise Analysis
- 3.3.6 Cost Component Breakdown
- 3.3.7 Monthly Cost Trend Analysis
- 3.3.8 Advanced Analysis using Heatmap
- 3.3.9 Anomaly Detection
- 3.3.10 Automated Insights Generation
- 3.4 Testing
- 3.5 Graphical User Interface
- 3.6 Customer Testing
- 3.7 Evaluation
 - 3.7.1 Strengths of the System
 - 3.7.2 Limitations of the System
 - 3.7.3 Overall Evaluation

4 SNAPSHOTS OF THE PROJECT

5 CONCLUSION

6 FURTHER DEVELOPMENT OR RESEARCH

7 REFERENCES

8 APPENDIX

Chapter 1

EXECUTIVE SUMMARY

This project presents the development of a Cloud Cost Intelligence system designed specifically for healthcare environments. As healthcare organizations increasingly rely on cloud infrastructure for storing and processing data, managing the associated costs has become an important concern. Uncontrolled cloud usage can lead to unexpected expenses, making it necessary to monitor and analyze cost patterns effectively.

The proposed system focuses on analyzing cloud billing data to understand how costs are distributed across different services and time periods. It also identifies unusual spikes in expenditure and highlights areas where optimization is possible. By doing so, the system helps in improving cost awareness and supports better decision-making.

The implementation is carried out using Python and its supporting libraries. Pandas is used for handling and analyzing data, Matplotlib is used for generating visual representations, and Streamlit is used to build an interactive dashboard. Users can upload billing data in a simple format and view clear visual insights such as cost trends, service-wise distribution, and anomaly detection results.

Overall, the system is designed to be easy to use, transparent in its analysis, and practical for real-world applications. It provides a straightforward approach to understanding cloud expenses in healthcare settings and helps in identifying opportunities for cost optimization without requiring complex technical knowledge.

Chapter 2

BACKGROUND

Cloud computing has significantly transformed the healthcare sector by enabling organizations to store, process, and access large volumes of data in a flexible and scalable manner. Modern healthcare systems rely heavily on cloud platforms for managing electronic health records, running diagnostic applications, supporting telemedicine services, and ensuring seamless data sharing across departments. This shift has improved efficiency, accessibility, and overall quality of healthcare services.

However, along with these advantages, cloud adoption has introduced new challenges, particularly in the area of cost management. Healthcare organizations typically utilize a wide range of cloud services, including compute resources, data storage systems, APIs, and network data transfer. Since these services operate on a usage-based pricing model, even small inefficiencies or unmonitored usage can result in a significant increase in operational expenses over time.

In many cases, the complexity of cloud billing structures makes it difficult for organizations to clearly understand where and how their resources are being consumed. Costs may vary depending on service type, time of usage, or seasonal demand, and without proper analysis, identifying the root cause of high expenditure becomes challenging. This can lead to budget overruns and reduced cost efficiency.

To address these issues, there is a growing need for systems that can provide clear insights into cloud usage and spending patterns. This project focuses on developing a Cloud Cost Intelligence system that analyzes cloud billing data in a structured manner. The system helps in identifying cost distribution, detecting unusual spending behavior, and highlighting areas where optimization can be applied. By improving visibility and understanding of cloud expenses, the system supports better financial planning and resource utilization in healthcare environments.

2.1 AIM AND OBJECTIVES

2.1.1 Aim

The primary aim of this project is to design and develop a Cloud Cost Intelligence system that can effectively analyze and optimize cloud expenses in healthcare environments. The system is intended to provide better visibility into cloud usage and support informed decision-making for cost management.

2.1.2 Objectives

The key objectives of this project are outlined below:

- To systematically analyze cloud billing data in order to understand overall spending patterns
- To identify how costs are distributed across different cloud services and departments
- To detect unusual or unexpected increases in cost using anomaly detection techniques
- To present cost trends over time through clear and meaningful visualizations
- To generate automated insights that highlight important observations in the data
- To provide practical recommendations that can help in reducing unnecessary cloud expenditure

2.2 TECHNOLOGIES USED

The development of this project involves the use of several technologies and tools that support data analysis, visualization, and application development.

2.2.1 Python

Python is used as the core programming language for developing the entire system. It provides a simple and readable syntax, making it suitable for rapid development and integration of different modules. Python also offers extensive support for data analysis, machine learning, and web-based visualization through its rich ecosystem of libraries.

2.2.2 Pandas

Pandas is used for data manipulation and analysis. It helps in handling structured data efficiently by providing data frames that support operations such as filtering, grouping, aggregation, and transformation. In this project, Pandas is mainly used for processing the cloud usage dataset, performing feature engineering, and preparing data for analysis.

2.2.3 NumPy

NumPy is used for numerical computations and array operations. It provides efficient handling of large datasets and supports mathematical operations required during data processing. NumPy works closely with Pandas to improve performance and enable faster computations.

2.2.4 Scikit-learn

Scikit-learn is used for implementing machine learning algorithms. In this project, it is primarily used to apply the **Isolation Forest algorithm** for anomaly detection. This helps in identifying unusual cloud cost patterns and abnormal usage behavior within the dataset.

2.2.5 Streamlit

Streamlit is used to develop the interactive dashboard for the system. It allows quick creation of web-based applications using Python without requiring advanced frontend knowledge. In this project, Streamlit is used to display key metrics, charts, anomaly visualizations, and recommendations in a user-friendly interface.

2.2.6 Matplotlib

Matplotlib is used for data visualization. It is mainly used to create graphs such as scatter plots for anomaly detection and other visual representations of cost patterns. It helps users understand data insights more clearly.

CSV (Data Format)

The dataset is stored and processed in CSV format, which is lightweight, easy to handle, and widely used for data exchange.

2.3 HARDWARE ARCHITECTURE

The system is designed to run on a standard personal computer without requiring any specialized hardware. This ensures that the solution is accessible and easy to deploy.

Minimum system requirements include:

- Processor: Intel i3 or equivalent
- RAM: Minimum 4 GB
- Storage: 500 GB HDD or SSD

Since the system is not computationally intensive, it can run efficiently on basic hardware configurations.

2.4 SOFTWARE ARCHITECTURE

The system follows a modular architecture, where each component is responsible for a specific function. This design improves maintainability, flexibility, and scalability of the system.

The architecture consists of the following layers:

- **Data Input Layer**
This layer handles the uploading of cloud billing data in CSV format through the Streamlit interface. It ensures that users can easily provide input data to the system.
- **Data Processing Layer**
In this layer, Pandas is used to clean, organize, and prepare the dataset. Data validation is also performed to ensure that all required fields are present and correctly formatted.
- **Analysis Layer**
This layer performs cost calculations, identifies patterns, and detects anomalies using a rule-based approach. Key metrics such as total cost, average cost, and peak values are computed in this stage.
- **Visualization Layer**
Matplotlib is used to generate graphical representations such as pie charts, bar graphs, and line plots. These visualizations help in understanding cost distribution and trends effectively.
- **Interface Layer**
The Streamlit dashboard acts as the user interface, displaying results in an interactive manner. It allows users to upload data, view outputs, and interpret insights easily.

Chapter 3

SYSTEM OVERVIEW

The proposed system is designed to provide a simple and effective way to analyze cloud billing data in healthcare environments. It operates through an interactive dashboard where users can easily upload their cloud cost data in CSV format. Once the dataset is uploaded, the system performs a series of processing steps in a structured manner. Initially, data validation is carried out to ensure that all required fields are present and correctly formatted. This helps in avoiding errors during further analysis. After validation, the system performs cost analysis by calculating key metrics such as total cost, average cost, and service-wise cost distribution. These calculations help in understanding how cloud resources are being utilized.

The system then generates visual representations of the data, including charts and graphs, which make it easier to interpret cost trends and patterns over time. Visualization plays an important role in simplifying complex data into understandable insights.

In addition to analysis, the system includes an anomaly detection mechanism that identifies unusual spikes or deviations in cost. This helps in detecting unexpected expenses that may require further investigation.

Finally, the system generates automated insights based on the analysis results. These insights highlight important observations and provide basic recommendations for cost optimization.

All the outputs are presented through a user-friendly dashboard, allowing users to interact with the data and gain a clear understanding of their cloud spending.

3.1 REQUIREMENTS

The requirements of the system are categorized into functional, user, and environmental requirements. These requirements define the expected behavior and conditions under which the system operates.

3.1.1 Functional Requirements

The functional requirements describe the core operations that the system must perform:

- The system should allow users to upload cloud billing data in CSV format for analysis.
- It should process the uploaded data and display key cost metrics such as total cost, average cost, and peak usage.
- The system must generate graphical visualizations including charts and graphs to represent cost distribution and trends.
- It should be capable of detecting anomalies in cost patterns using predefined logic.
- The system should generate automated insights based on the analysis to help users understand important cost-related observations.

3.1.2 User Requirements

The system is designed to be user-friendly and accessible, even for users with minimal technical knowledge:

- The interface should be simple and easy to navigate.
- Visualizations should be clear and easy to interpret.
- The data upload process should be straightforward without requiring complex steps.

3.1.3 Environmental Requirements

The system requires a basic software environment for execution:

- Python should be installed on the system.
- Required libraries such as Pandas, Matplotlib, and Streamlit must be available.
- The system should run in a standard development environment such as VS Code or any Python-supported platform.

3.2 DESIGN AND ARCHITECTURE

The system is designed using a pipeline-based architecture, where data flows through a sequence of well-defined stages. This approach ensures clarity in processing and makes the system easy to understand and maintain.

The overall workflow of the system can be represented as:

Input → Processing → Analysis → Visualization → Output

- **Input Stage**
In this stage, the user uploads cloud billing data in CSV format through the dashboard interface.
- **Processing Stage**
The uploaded data is cleaned and validated to ensure that all required fields are present and correctly structured.
- **Analysis Stage**
The system performs cost-related computations such as total cost, average cost, service-wise distribution, and anomaly detection.
- **Visualization Stage**
The processed data is converted into graphical formats such as charts and plots to make the information easier to interpret.
- **Output Stage**
The final results, including metrics, graphs, anomalies, and insights, are displayed through the dashboard.

Each module in the system is designed to function independently. This modular design improves flexibility, allowing individual components to be updated or extended without affecting the entire system. It also supports scalability for future enhancements.

3.3 IMPLEMENTATION

The implementation of the proposed Cloud Cost Intelligence system is carried out using Python, focusing on simplicity, clarity, and practical usability. The system is designed as an interactive dashboard that allows users to upload cloud billing data and obtain meaningful insights through automated analysis and visualization.

The implementation is divided into several functional components, each responsible for a specific task in the overall workflow.

3.3.1 Data Input and Validation

The system begins with a file upload feature implemented using Streamlit. Users can upload cloud billing data in CSV format through the interface. Once the file is uploaded, the system performs validation to ensure that all required columns, such as service type, cost, and month, are present.

This step is important to avoid errors during processing and ensures that the data is in a suitable format for further analysis.

3.3.2 Cost Analysis and Aggregation

After validation, the system processes the dataset using the Pandas library. Various cost-related metrics are calculated, including total cost, average cost, and peak cost values. The data is also grouped based on service type and departments to understand how cloud expenses are distributed.

This aggregation helps in identifying which services or departments contribute the most to overall costs.

3.3.3 Service-wise Cost Distribution

To visualize how costs are distributed across different cloud services, a pie chart is generated.

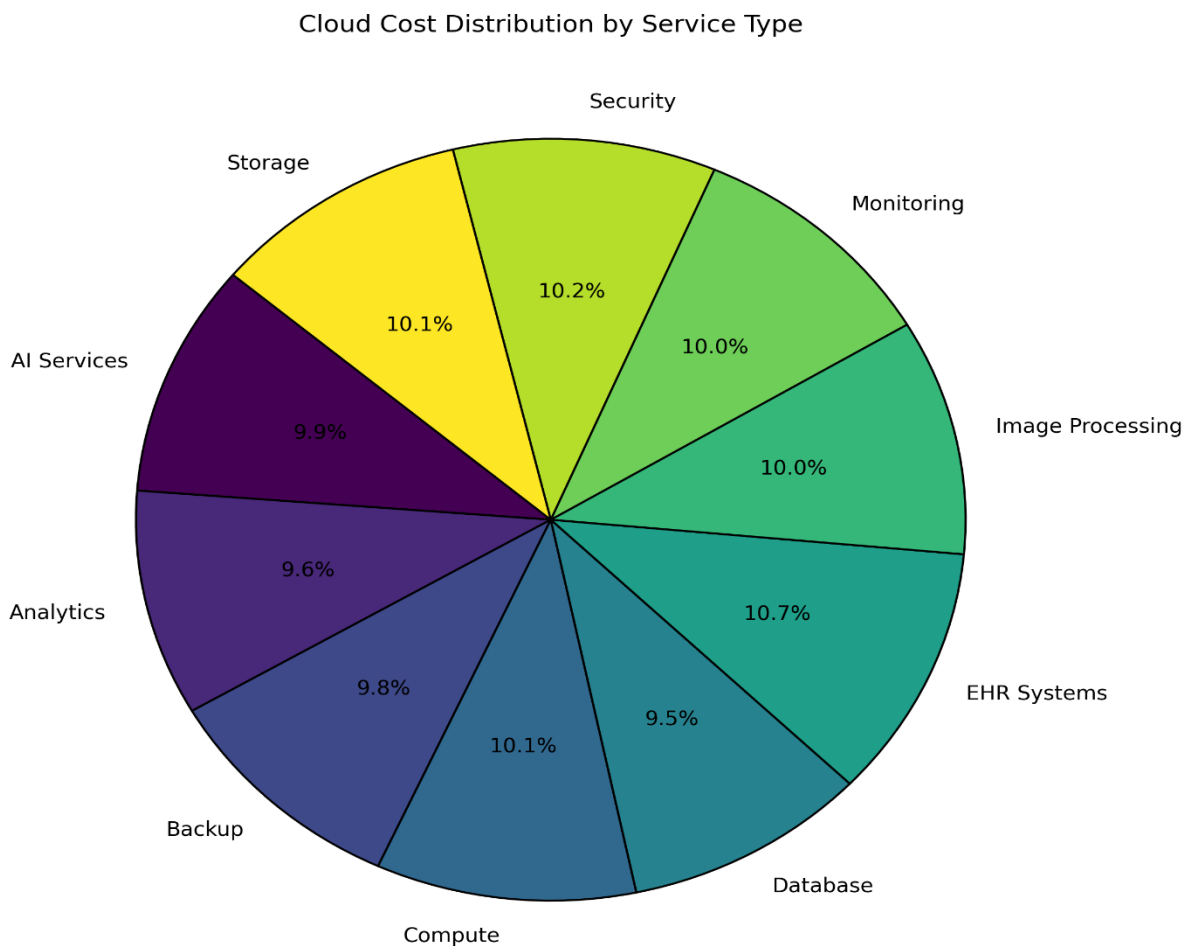


Figure 3.1: Service-wise Cloud Cost Distribution

This figure shows the proportion of cost contributed by services such as compute, storage, database, and AI services. It helps in quickly identifying major cost contributors.

3.3.4 Service Usage Analysis

The system also analyzes how frequently different cloud services are used.

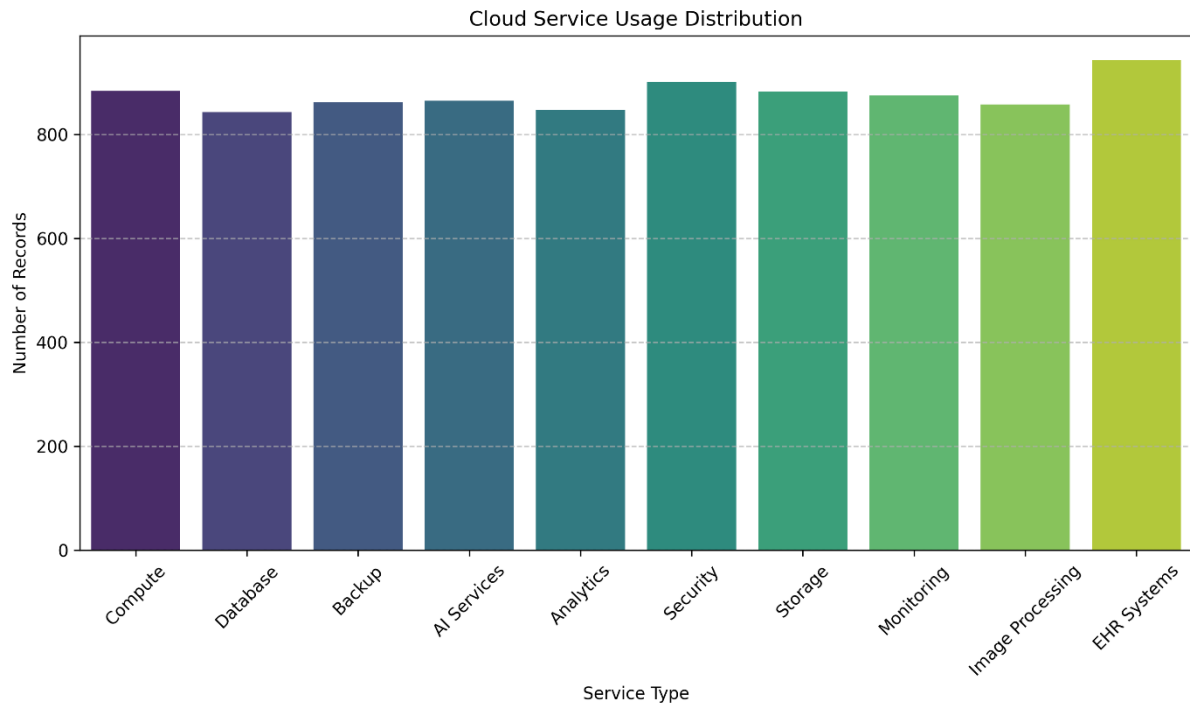


Figure 3.2: Cloud Service Usage Distribution

This bar chart represents the number of records associated with each service type, providing insight into usage patterns.

3.3.5 Department-wise Analysis

To understand how different hospital departments utilize cloud resources, a department-wise analysis is performed.

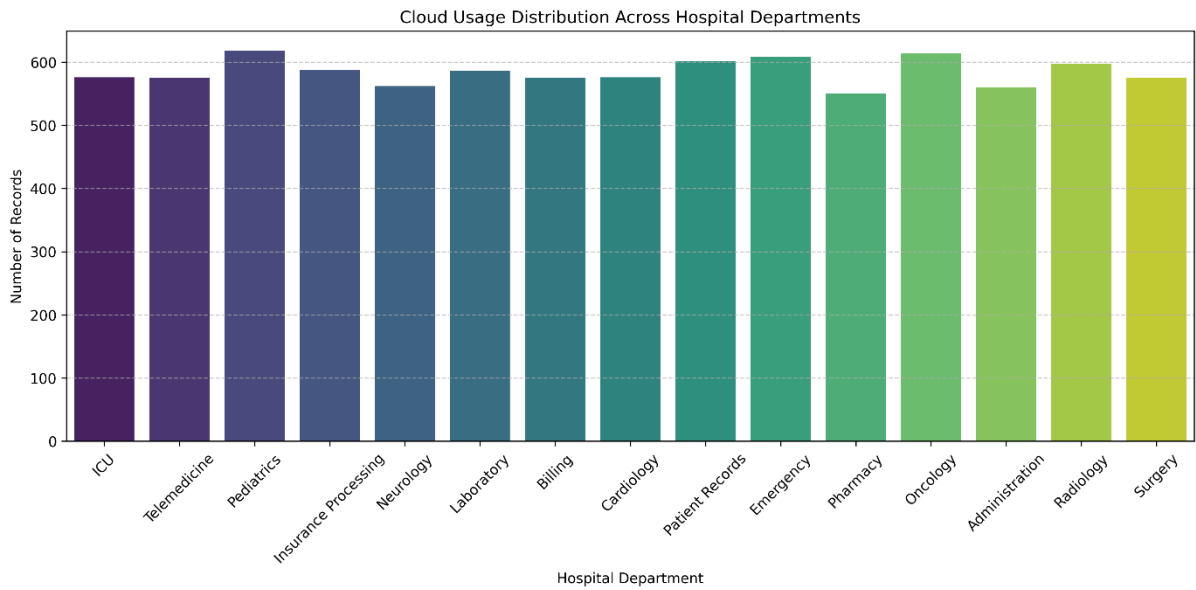


Figure 3.3: Cloud Usage Across Hospital Departments

3.3.6 Cost Component Breakdown

The system further breaks down the total cost into individual components such as compute, storage, transfer, and API costs.

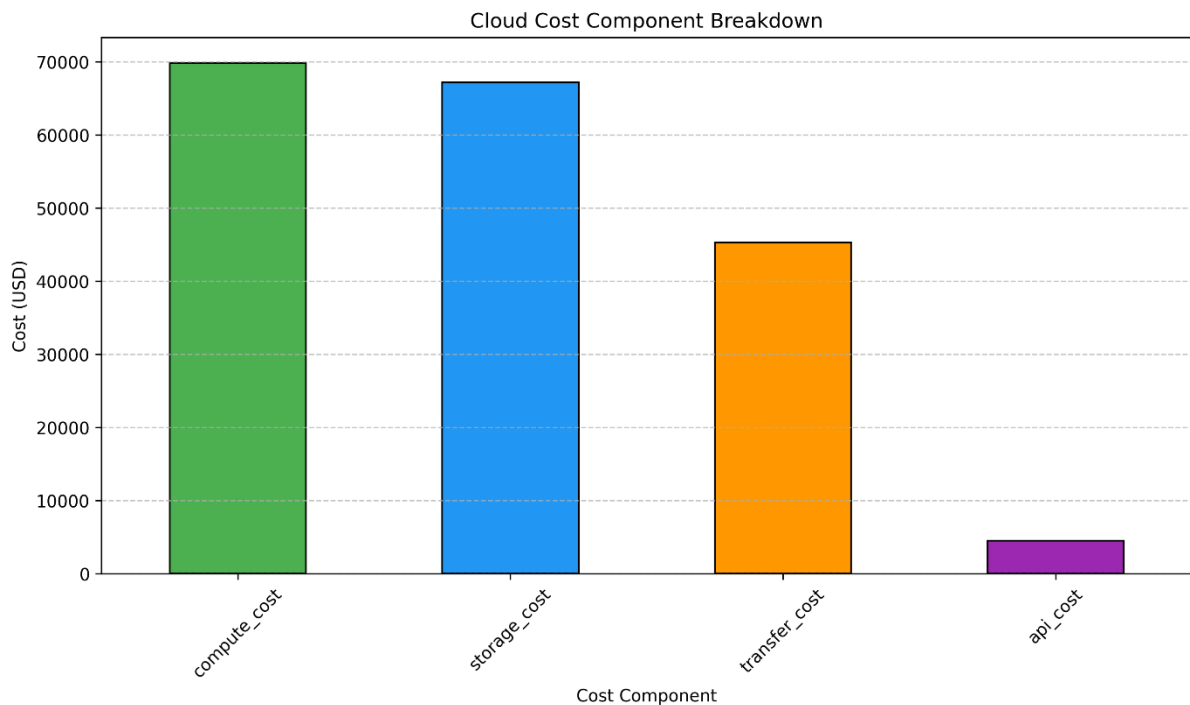


Figure 3.4: Cloud Cost Component Breakdown

This helps in identifying which component contributes the most to overall expenditure, enabling targeted optimization. This visualization highlights the variation in cloud usage across departments such as ICU, radiology, billing, and administration.

3.3.7 Monthly Cost Trend Analysis

To analyze how cloud costs vary over time, a monthly trend graph is generated.

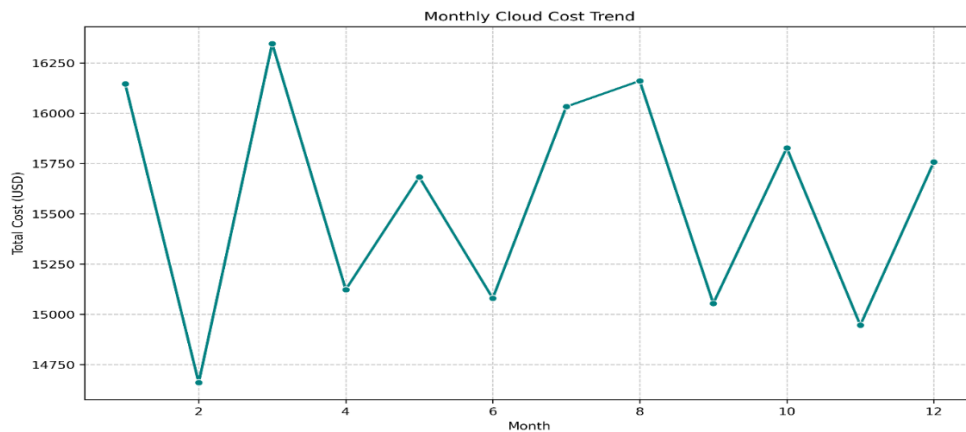


Figure 3.5: Monthly Cloud Cost Trend

This line graph helps in identifying patterns such as increasing costs, seasonal variations, or sudden drops.

3.3.8 Advanced Analysis using Heatmap

The relationship between departments and services is visualized using a heatmap.

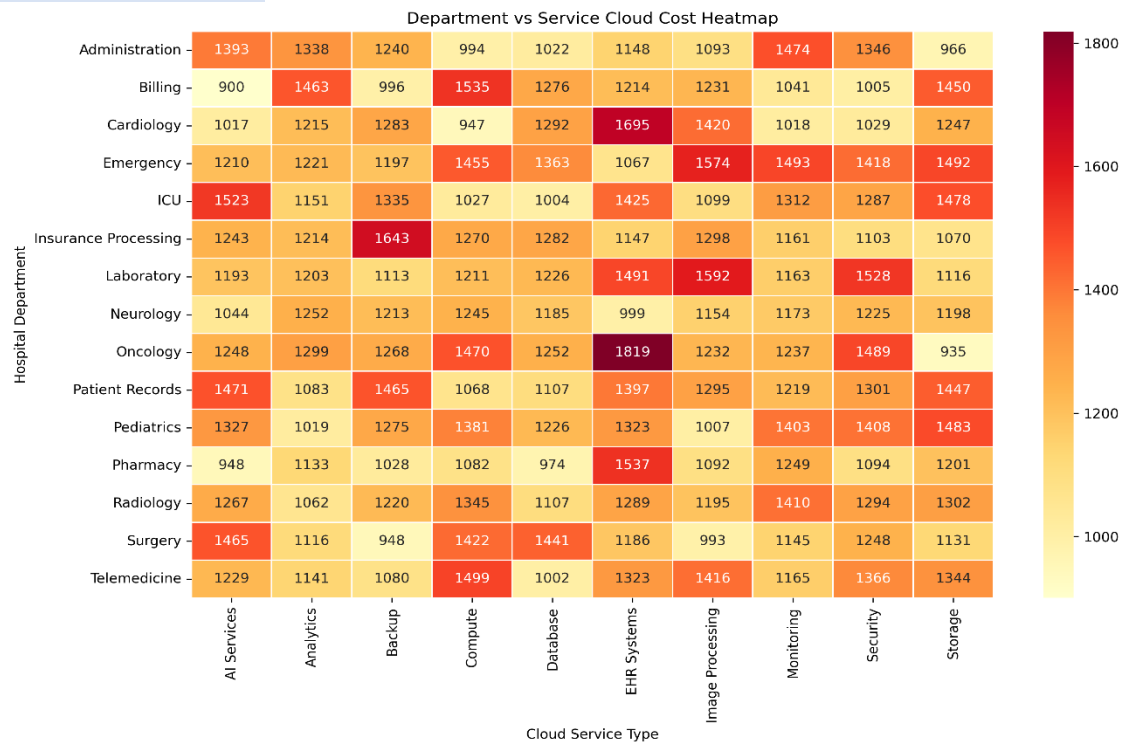


Figure 3.6: Department vs Service Cloud Cost Heatmap

This figure provides a detailed view of how different departments consume various cloud services, helping in deeper analysis.

3.3.9 Anomaly Detection

The system implements a rule-based anomaly detection mechanism using a threshold approach ($\text{mean} + 2 \times \text{standard deviation}$). This helps in identifying unusual cost spikes.

3.3.10 Automated Insights Generation

Based on the analysis, the system automatically generates insights such as:

- Highest cost contributing service
- Peak cost month
- Presence of anomalies
- Major cost components

These insights are displayed directly in the dashboard, helping users make informed decisions without manual analysis.

3.4 TESTING

The system was tested using multiple sample datasets to ensure that all functionalities work correctly and produce reliable results. Testing was carried out at different stages of the system to verify both data processing and output accuracy.

The following aspects were tested:

- **Data Validation Checks**

The system was tested with both valid and invalid datasets to ensure that it correctly identifies missing or incorrect columns. Appropriate error messages were displayed when the required data format was not met.

- **Accuracy of Calculations**

Cost-related metrics such as total cost, average cost, and peak values were verified manually and compared with system outputs to ensure correctness.

- **Graphical Output Verification**

All visualizations, including pie charts, bar graphs, and line plots, were checked to confirm that they accurately represent the underlying data and update correctly based on input.

- **Anomaly Detection Verification**

The anomaly detection logic was tested by introducing datasets with artificially high cost values. The system successfully identified these as anomalies using the defined threshold method. The effectiveness of the anomaly detection mechanism is illustrated in the figure below.

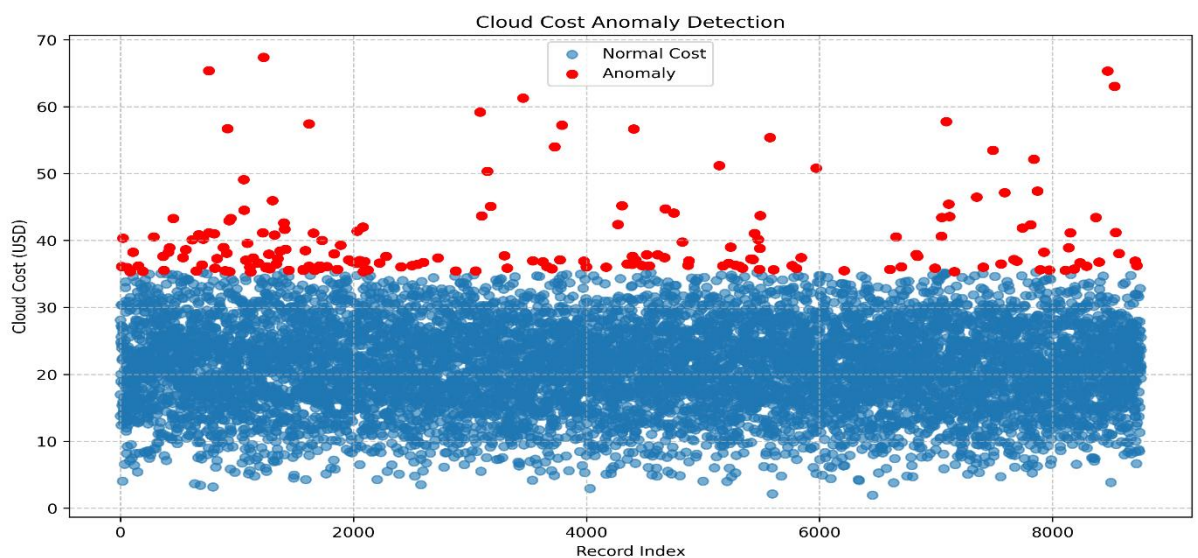


Figure 3.7: Cloud Cost Anomaly Detection Output

The graph shows normal cost values and detected anomalies. The red points indicate abnormal spikes in cloud cost, confirming that the system successfully identifies unusual patterns based on the defined threshold.

- **System Performance**

The system was tested for responsiveness and was able to process datasets efficiently without noticeable delay.

Overall, the testing process confirmed that all modules of the system are functioning as expected and producing consistent and accurate results.

3.5 GRAPHICAL USER INTERFACE

The Graphical User Interface (GUI) of the system is developed using Streamlit, which enables the creation of an interactive and user-friendly web-based dashboard. The interface is designed with simplicity in mind so that users can easily interact with the system without requiring advanced technical knowledge.

The main features of the GUI include:

- **File Upload Option**

Users are provided with an option to upload cloud billing data in CSV format. The upload process is straightforward and ensures quick data input into the system.

- **KPI Metrics Display**

Key performance indicators such as total cost, average cost, peak cost, and anomaly count are displayed prominently. This allows users to quickly understand the overall cost status.

- **Graphical Visualizations**

The dashboard presents various charts including pie charts, bar graphs, and line plots. These visual elements help in understanding cost distribution across services, departments, and time periods.

- **Insights and Recommendations Section**

The system automatically generates insights based on the analyzed data. Recommendations are also provided to help users make better decisions regarding cost optimization.

The user interface of the system is illustrated in the figure below.

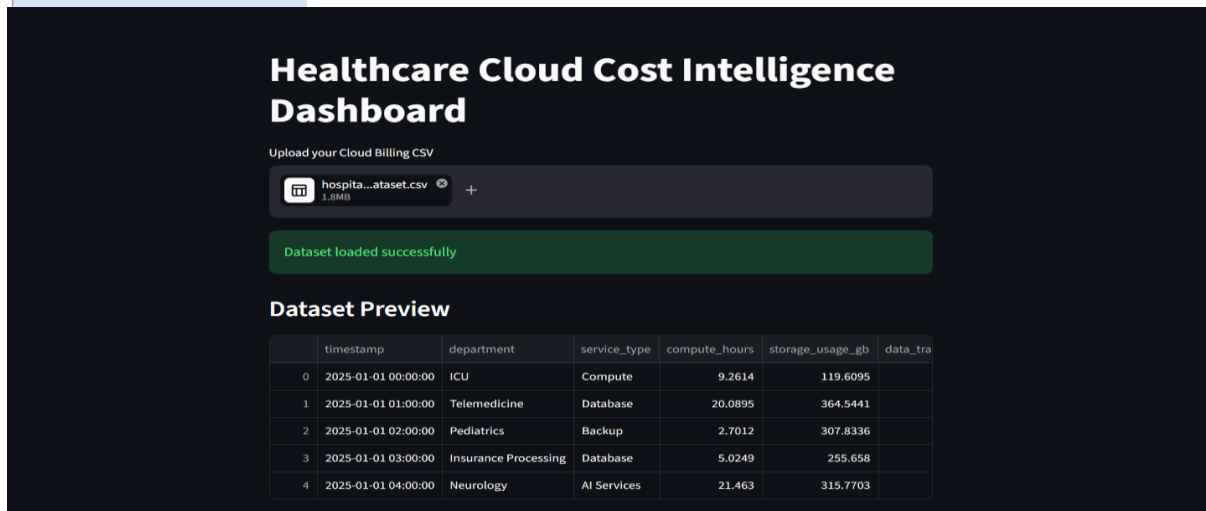


Figure 3.8: Cloud Cost Intelligence Dashboard Interface

The dashboard displays key metrics, visualizations, and insights in a structured format. It allows users to upload data and instantly view analysis results, making the system interactive and easy to use.

The interface is organized in a clear and structured manner, ensuring that users can easily navigate between different sections. Overall, the GUI enhances the usability of the system by presenting complex data in a simple and visually understandable format.

3.6 CUSTOMER TESTING

Customer testing was carried out to evaluate the usability and effectiveness of the developed Cloud Cost Intelligence system. The testing was performed using a sample healthcare cloud billing dataset.

During the testing process, users interacted with the system by uploading the dataset and exploring various features available in the dashboard. This included viewing cost metrics, analyzing visualizations, and understanding the generated insights.

Observations from Customer Testing

- Users were able to upload the dataset easily without any errors
- The interface was simple and easy to understand
- Graphical representations helped users interpret cost distribution clearly
- The anomaly detection feature successfully highlighted unusual cost values
- The insights generated by the system were helpful in understanding spending patterns

User Feedback

Users found the system easy to use and effective in presenting cloud cost information. Even with a single dataset, the system was able to provide meaningful insights and clear visual outputs. The testing results indicate that the system is user-friendly and performs effectively in analyzing and visualizing cloud cost data. It meets the expected requirements and provides a smooth user experience.

3.7 EVALUATION

The developed Cloud Cost Intelligence system was evaluated based on its performance, usability, and effectiveness in analyzing healthcare cloud billing data.

The system successfully performs cost analysis by calculating important metrics such as total cost, average cost, and service-wise cost distribution. The results obtained from the system were consistent and accurate when compared with manual calculations, indicating the reliability of the implementation.

The visualization component plays a key role in improving the understanding of data. Graphs such as pie charts, bar charts, and line plots help in clearly representing cost distribution and trends over time. These visualizations make it easier for users to interpret complex data without requiring advanced technical knowledge.

The anomaly detection feature, implemented using a rule-based threshold approach, is effective in identifying unusual spikes in cloud costs. This helps in detecting unexpected expenses and enables users to take necessary actions.

From a usability perspective, the system performs well. The Streamlit-based dashboard provides a simple and interactive interface where users can upload data and view results easily. The workflow is straightforward, and users can navigate through the system without difficulty.

3.7.1 Strengths of the System

- The system provides a simple and user-friendly interface
- It generates clear and meaningful visualizations
- Cost calculations are accurate and reliable
- The anomaly detection feature helps identify unusual cost patterns
- Automated insights assist users in understanding cloud expenses

3.7.2 Limitations of the System

- The anomaly detection approach is rule-based and may not detect complex patterns
- The system currently works only with static datasets

- Advanced machine learning techniques are not implemented
- Performance with very large datasets is not evaluated

3.7.3 Overall Evaluation

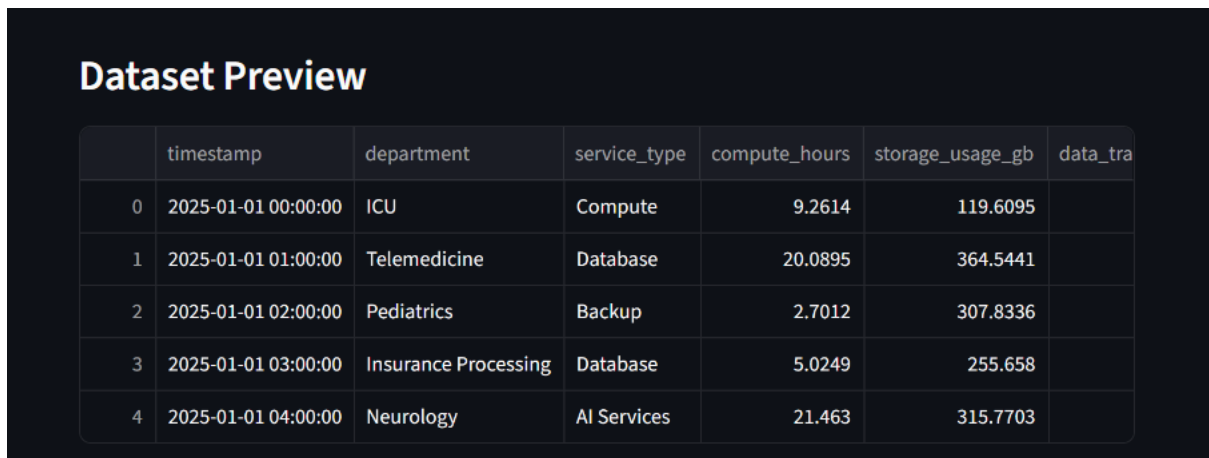
Overall, the system effectively analyzes cloud cost data and provides useful insights for decision-making. It achieves the intended objectives by combining data analysis, visualization, and anomaly detection in a simple and practical manner. The system serves as a strong foundation for further improvements such as real-time monitoring and integration of advanced analytical techniques.

Chapter 4

SNAPSHOTS OF THE PROJECT

This section presents the visual outputs generated by the Cloud Cost Intelligence system through the Streamlit dashboard. These snapshots demonstrate how the system processes cloud billing data and converts it into meaningful insights using various visualizations.

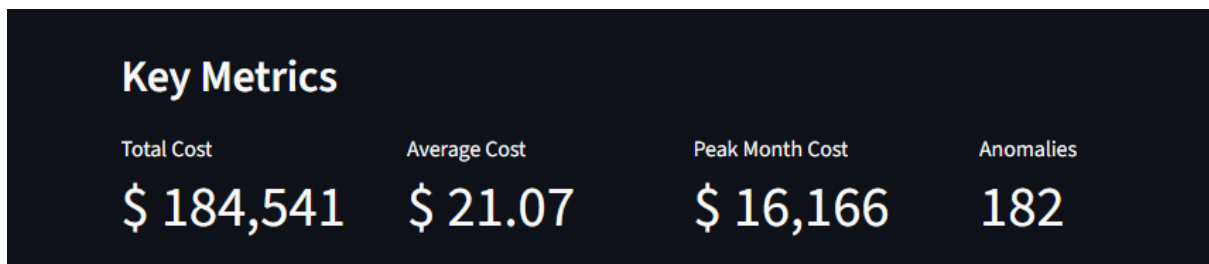
Figure 4.1: Dataset Preview



	timestamp	department	service_type	compute_hours	storage_usage_gb	data_tra
0	2025-01-01 00:00:00	ICU	Compute	9.2614	119.6095	
1	2025-01-01 01:00:00	Telemedicine	Database	20.0895	364.5441	
2	2025-01-01 02:00:00	Pediatrics	Backup	2.7012	307.8336	
3	2025-01-01 03:00:00	Insurance Processing	Database	5.0249	255.658	
4	2025-01-01 04:00:00	Neurology	AI Services	21.463	315.7703	

This figure shows a preview of the uploaded dataset. It allows users to verify the data and ensure that it is correctly loaded before performing further analysis.

Figure 4.2: Key Metrics Dashboard

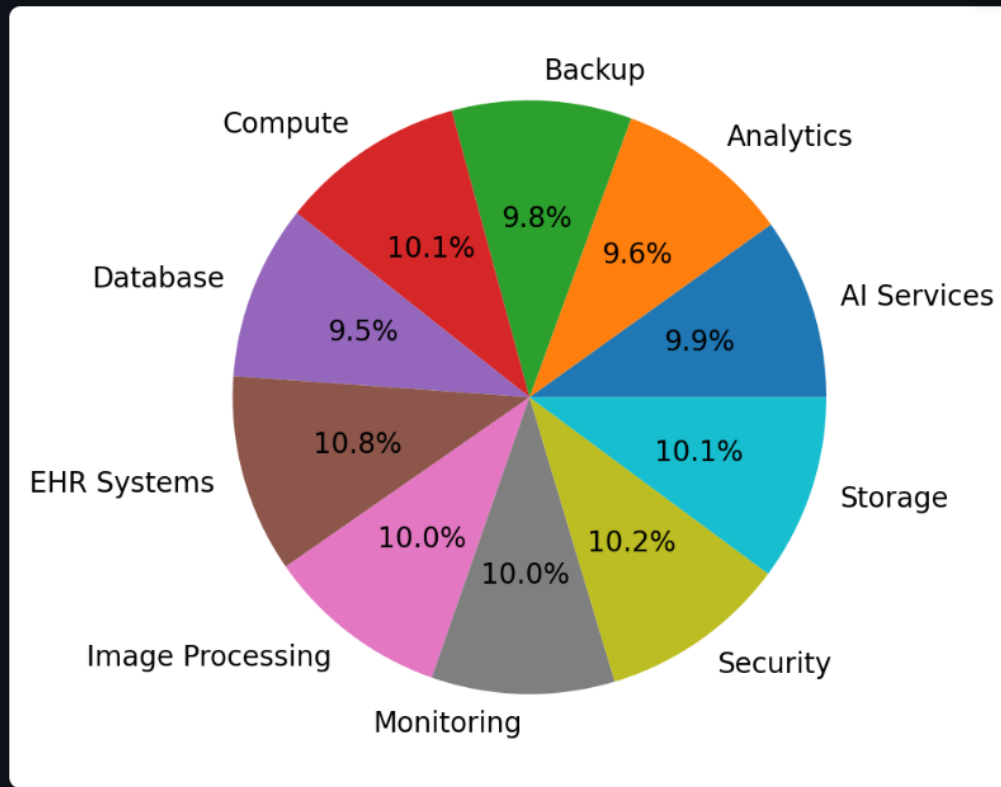


Total Cost	Average Cost	Peak Month Cost	Anomalies
\$ 184,541	\$ 21.07	\$ 16,166	182

This section displays important metrics such as total cost, average cost, peak month cost, and the number of anomalies detected. It provides a quick overview of the overall cost status.

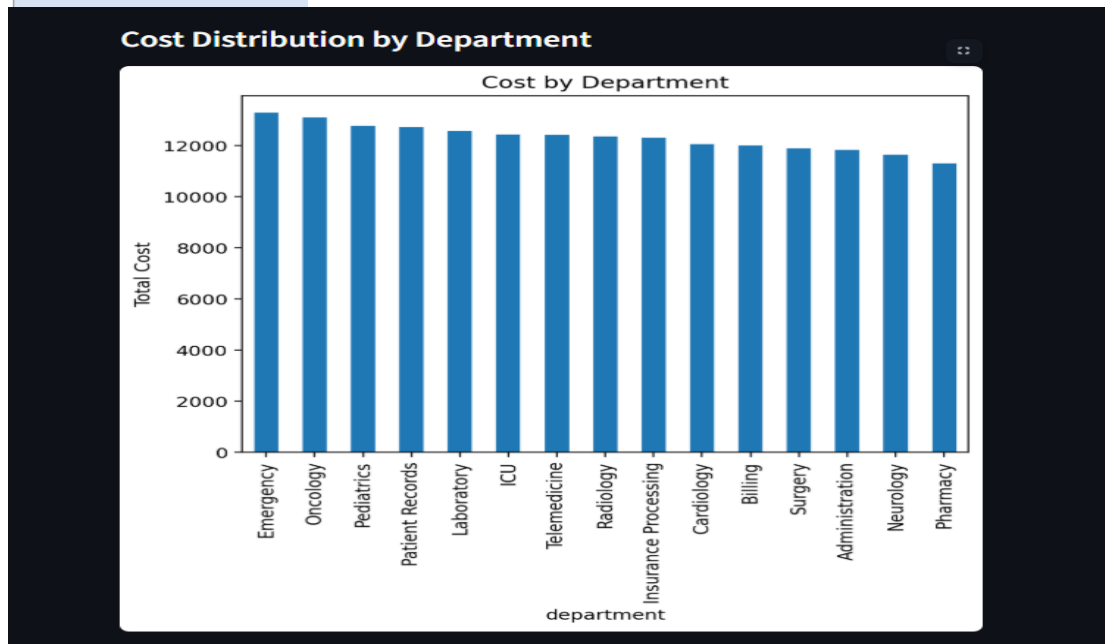
Figure 4.3: Service-wise Cost Distribution

Cost Distribution by Service



This figure illustrates how cloud costs are distributed across different services. It helps in identifying which services contribute the most to total expenditure.

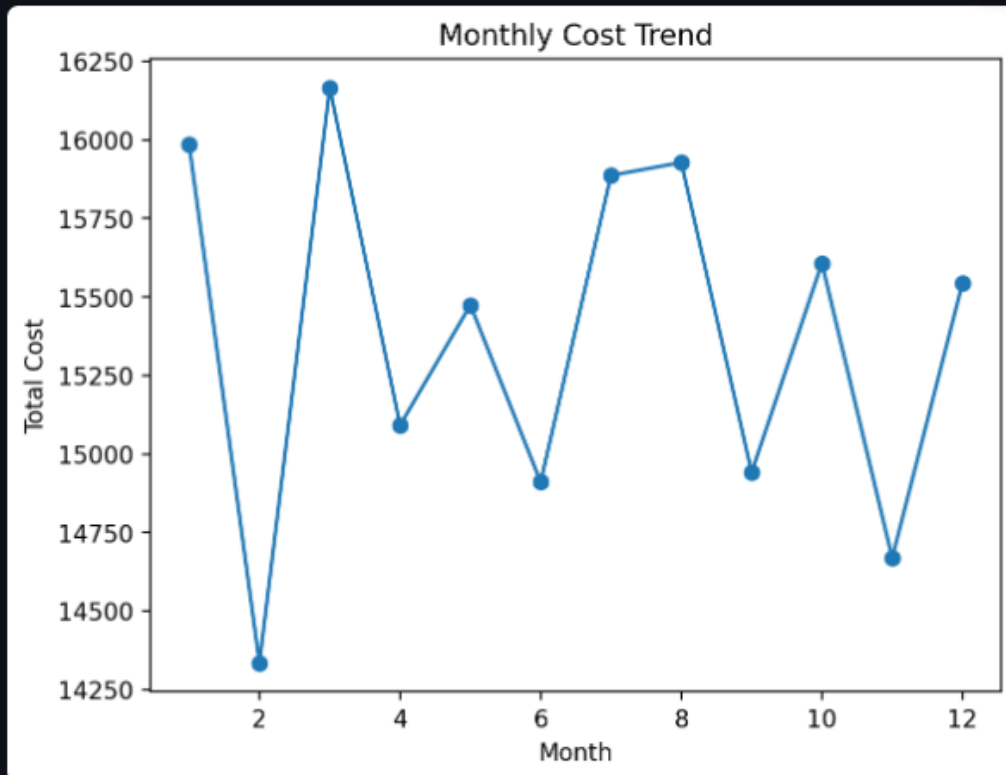
Figure 4.4: Department-wise Cost Distribution



This chart shows how cloud costs vary across different departments, making it easier to understand department-level usage.

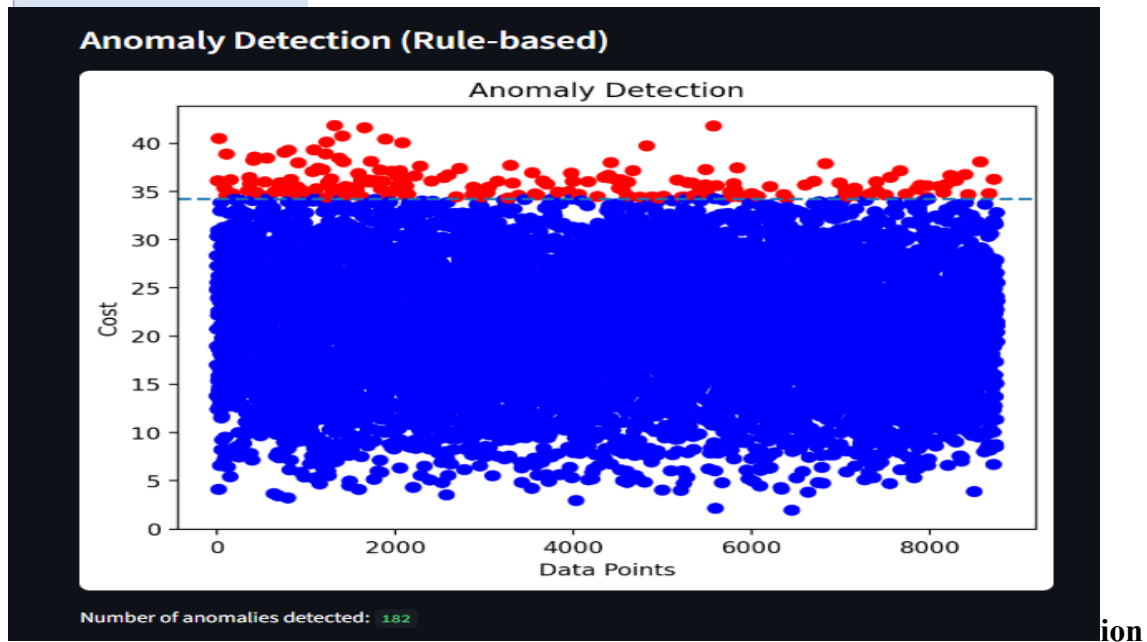
Figure 4.5: Monthly Cloud Cost Trend

Monthly Cost Trend



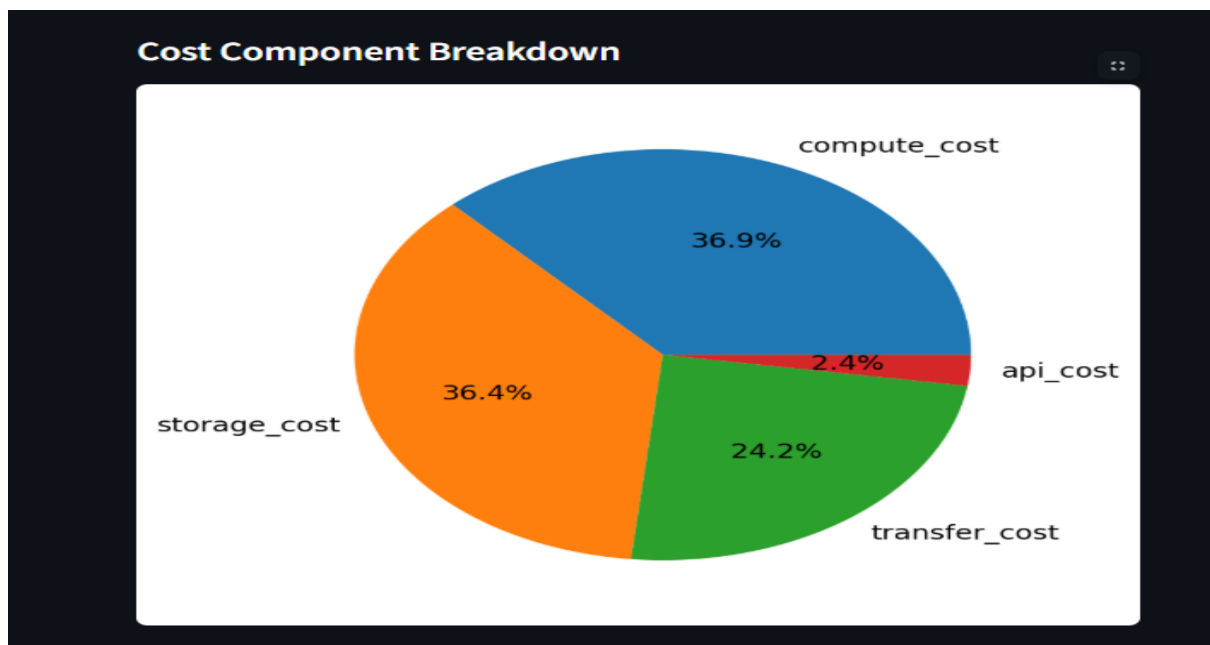
This graph represents the variation in cloud cost over different months. It helps in identifying trends, fluctuations, and possible seasonal patterns.

Figure 4.6: Cloud Cost Anomaly Detect



This visualization highlights unusual cost values. Anomalies are clearly identified, allowing users to detect unexpected spending patterns.

Figure 4.7: Cost Component Breakdown



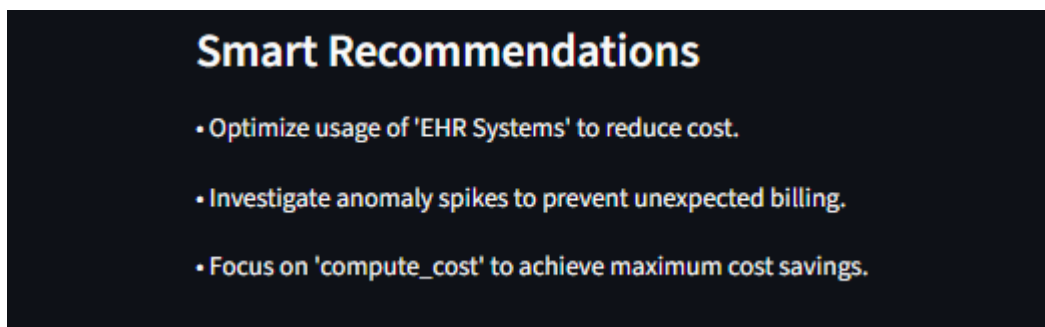
This figure shows how total cost is divided among different components such as compute, storage, data transfer, and API usage.

Figure 4.8: Auto-generated Insights



This section displays automatically generated insights based on the analyzed data, helping users quickly understand key observations.

Figure 4.9: Smart Recommendations



This figure presents system-generated recommendations that guide users in optimizing their cloud costs.

Chapter 5

CONCLUSION

The Cloud Cost Intelligence system developed in this project successfully demonstrates how cloud billing data can be analyzed to generate meaningful insights in a healthcare environment. The system focuses on simplifying complex cost data and presenting it in a clear and understandable manner.

Through the implementation of data processing, visualization, and anomaly detection techniques, the system is able to identify cost patterns, highlight unusual spending behavior, and provide useful recommendations. The use of graphical representations such as charts and trends helps users quickly interpret data without requiring advanced technical knowledge.

The Streamlit-based dashboard enhances the usability of the system by offering an interactive interface where users can easily upload data and view results. This makes the system practical and accessible for real-world applications.

Overall, the project achieves its objective of building a simple and effective solution for monitoring and analyzing cloud costs in healthcare. It provides a strong foundation for further improvements and can be extended to support more advanced features in the future.

Chapter 6

FURTHER DEVELOPMENT OR RESEARCH

The current system provides a basic and effective approach for analyzing cloud cost data in healthcare environments. However, there are several areas where the system can be further enhanced to improve its functionality and real-world applicability.

One possible improvement is the integration of advanced machine learning techniques for anomaly detection and cost prediction. Instead of relying on a rule-based approach, models such as Isolation Forest or regression techniques can be used to identify complex patterns and forecast future cloud expenses more accurately.

Another important enhancement is the incorporation of real-time data processing. Currently, the system works with static datasets, but integrating cloud APIs would allow continuous monitoring of cloud usage and costs. This would enable organizations to take immediate actions when abnormal spending is detected.

The system can also be extended to include more detailed user access control and role-based dashboards. This would allow different stakeholders, such as administrators and financial analysts, to view customized reports based on their requirements.

In addition, scalability can be improved by optimizing the system to handle large datasets efficiently. This would make the solution more suitable for large healthcare organizations with high volumes of cloud usage.

Further research can also focus on integrating predictive analytics to estimate future costs based on historical data. This would help in better budgeting and resource planning.

Overall, the project provides a strong foundation, and with these enhancements, it can be developed into a more advanced and production-ready cloud cost management system.

Chapter 7

REFERENCES

The following resources were referred to during the development of this project:

1. Python Software Foundation, *Python Documentation*. Available at: <https://docs.python.org/>
2. McKinney, W., *Python for Data Analysis*. O'Reilly Media, 2017.
3. The Pandas Development Team, *Pandas Documentation*. Available at: <https://pandas.pydata.org/docs/>
4. Hunter, J. D., *Matplotlib Documentation*. Available at: <https://matplotlib.org/stable/index.html>
5. Streamlit Inc., *Streamlit Documentation*. Available at: <https://docs.streamlit.io/>
6. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 2019.
7. Scikit-learn Developers, *Scikit-learn Documentation*. Available at: <https://scikit-learn.org/stable/>
8. AWS, *Cloud Cost Management and Billing Documentation*. Available at: <https://aws.amazon.com/aws-cost-management/>
9. Google Cloud, *Cloud Billing Documentation*. Available at: <https://cloud.google.com/billing/docs>

Chapter 8

APPENDIX

This section provides additional supporting details related to the implementation and execution of the Cloud Cost Intelligence system.

A. Sample Dataset Structure

The system uses a CSV file as input. A sample structure of the dataset is shown below:

timestamp	department	service_type	compute_hours	storage_usage_gb	data_transfer_gb
01-01-2025 00:00	ICU	Compute	9.261448255	119.6094547	74.73098155
01-01-2025 01:00	Telemedicine	Database	19.68448832	364.5440531	95.54443607
01-01-2025 02:00	Pediatrics	Backup	2.701153307	307.8336483	113.2254821
01-01-2025 03:00	Insurance Processing	Database	5.024927061	255.6580114	161.3612196
01-01-2025 04:00	Neurology	AI Services	21.46295568	315.7702606	46.17549362
01-01-2025 05:00	Laboratory	Database	19.76858398	112.0599372	82.75925043
01-01-2025 06:00	Pediatrics	AI Services	22.85653315	399.0299662	30.36590652
01-01-2025 07:00	Billing	Compute	23.97125079	474.5997745	19.77329846
01-01-2025 08:00	ICU	Analytics	9.552782653	446.6405406	122.9114995

api_requests	active_users	cpu_utilization	month	flu_season	viral_season	allergy_season
8992	122	26.58132332	1	TRUE	FALSE	FALSE
6122.625387	293	16.73833063	1	TRUE	FALSE	FALSE
4856	95	38.30237421	1	TRUE	FALSE	FALSE
2007	111	37.68588517	1	TRUE	FALSE	FALSE

1236	28	66.53286402	1	TRUE	FALSE	FALSE
6592.704538	72	81.43057965	1	TRUE	FALSE	FALSE
722	125	35.23499917	1	TRUE	FALSE	FALSE
7345	189	46.10570506	1	TRUE	FALSE	FALSE
2996	47	18.92120565	1	TRUE	FALSE	FALSE
injected_anomaly	compute_cost	storage_cost	transfer_cost	api_cost	cost	
0	5.556868953	3.588283642	3.736549078	0.8992	13.7809	
0	11.81069299	10.93632159	4.777221804	0.612263	28.1365	
0	1.620691984	9.23500945	5.661274106	0.4856	17.00258	
0	3.014956237	7.669740342	8.068060982	0.2007	18.95346	
0	12.87777341	9.473107818	2.308774681	0.1236	24.78326	
0	11.86115039	3.361798115	4.137962522	0.65927	20.02018	
0	13.71391989	11.97089899	1.518295326	0.0722	27.27531	
0	14.38275047	14.23799323	0.988664923	0.7345	30.34391	
0	5.731669592	13.39921622	6.145574973	0.2996	25.57606	
0	4.723482284	5.025105456	2.093637682	0.5759	12.41813	

This dataset contains information about cloud services, their associated costs, the time of usage, and the department responsible for the usage.

B. Key Code Snippet

The following code snippet shows the logic used for anomaly detection in the system:

```
threshold = df['cost'].mean() + 2 * df['cost'].std()
```

```
df['Anomaly'] = df['cost'].apply(
    lambda x: 'Anomaly' if x > threshold else 'Normal')
```

C. Tools and Libraries Used

- Python
- Pandas
- Matplotlib
- Streamlit
- Scikit-learn (used for extended analysis, if applicable)

D. Execution Steps

The following steps are used to run the system:

1. Activate the virtual environment
2. Install the required libraries
3. Run the application using the command:

```
streamlit run app.py
```

4. Upload the dataset through the dashboard

E. Complete Source Code

This section contains the complete implementation of the Cloud Cost Intelligence system developed as part of this project.

The source code includes:

- The main Streamlit application (app.py) used for data analysis and visualization
- The Jupyter Notebook used for initial data exploration and testing

The code is organized and structured to ensure readability and ease of understanding. Comments are included within the code to explain key functionalities and logic.

F.1 Streamlit Application (app.py)

```
import streamlit as st

import pandas as pd

import matplotlib.pyplot as plt

st.title("Healthcare Cloud Cost Intelligence Dashboard")

# ----- UPLOAD -----

uploaded_file = st.file_uploader("Upload your Cloud Billing CSV", type=["csv"])
```

if uploaded_file is not None:

```
df = pd.read_csv(uploaded_file)
```

```
st.success("Dataset loaded successfully")
```

```
# ----- VALIDATION -----
```

```
required_cols = ['service_type', 'cost', 'month']
```

```
if not all(col in df.columns for col in required_cols):
```

```
    st.error("Dataset must contain service_type, cost, and month columns")
```

```
    st.stop()
```

```
# ----- DATA PREVIEW -----
```

```
st.subheader("Dataset Preview")
```

```
st.write(df.head())
```

```
# ----- FIX TYPES -----
```

```
if 'seasonal_effect' in df.columns:
```

```
    df['seasonal_effect'] = df['seasonal_effect'].astype(bool)
```

```
# Ensure month sorting
```

```
month_order = list(range(1, 13))

monthly_cost = df.groupby('month')['cost'].sum().reindex(month_order)


# ----- KPI METRICS -----

st.subheader("Key Metrics")


total_cost = df['cost'].sum()

avg_cost = df['cost'].mean()

peak_month_cost = monthly_cost.max()


threshold = df['cost'].mean() + 2 * df['cost'].std()

anomaly_count = df[df['cost'] > threshold].shape[0]


col1, col2, col3, col4 = st.columns(4)


col1.metric("Total Cost", f"$ {total_cost:,.0f}")

col2.metric("Average Cost", f"$ {avg_cost:,.2f}")

col3.metric("Peak Month Cost", f"$ {peak_month_cost:,.0f}")

col4.metric("Anomalies", anomaly_count)


# ----- SERVICE COST -----

st.subheader("Cost Distribution by Service")
```

```
service_cost = df.groupby('service_type')['cost'].sum()

fig1, ax1 = plt.subplots()

service_cost.plot(kind='pie', autopct='%1.1f%%', ax=ax1)

ax1.set_ylabel("")

st.pyplot(fig1)

# ----- DEPARTMENT COST -----

st.subheader("Cost Distribution by Department")

if 'department' in df.columns:

    dept_cost = df.groupby('department')['cost'].sum().sort_values(ascending=False)

    fig_dept, ax_dept = plt.subplots()

    dept_cost.plot(kind='bar', ax=ax_dept)

    ax_dept.set_ylabel("Total Cost")

    ax_dept.set_title("Cost by Department")

    st.pyplot(fig_dept)

else:
```

```
st.warning("Department column not available")
```

```
# ----- MONTHLY TREND -----
```

```
st.subheader("Monthly Cost Trend")
```

```
fig_month, ax_month = plt.subplots()
```

```
ax_month.plot(monthly_cost.index, monthly_cost.values, marker='o')
```

```
ax_month.set_xlabel("Month")
```

```
ax_month.set_ylabel("Total Cost")
```

```
ax_month.set_title("Monthly Cost Trend")
```

```
st.pyplot(fig_month)
```

```
# ----- SEASONAL ANALYSIS -----
```

```
st.subheader("Seasonal Impact on Cost")
```

```
if 'seasonal_effect' in df.columns:
```

```
    seasonal_cost = df.groupby('seasonal_effect')['cost'].mean()
```

```
    seasonal_cost.index = ['Non-Seasonal', 'Seasonal']
```

```
fig_season, ax_season = plt.subplots()

colors = ['blue', 'orange']

ax_season.bar(seasonal_cost.index, seasonal_cost.values, color=colors)

ax_season.set_ylabel("Average Cost")

ax_season.set_title("Seasonal vs Non-Seasonal Cost")

st.pyplot(fig_season)

else:

    st.warning("Seasonal data not available")

# ----- ANOMALY DETECTION -----

st.subheader("Anomaly Detection (Rule-based)")

df['Anomaly'] = df['cost'].apply(

    lambda x: 'Anomaly' if x > threshold else 'Normal'

)

fig3, ax3 = plt.subplots()

colors = df['Anomaly'].map({
```

```

        'Normal': 'blue',

        'Anomaly': 'red'

    })

ax3.scatter(range(len(df)), df['cost'], c=colors)

ax3.axhline(threshold, linestyle='--')


ax3.set_xlabel("Data Points")

ax3.set_ylabel("Cost")

ax3.set_title("Anomaly Detection")


st.pyplot(fig3)


st.write("Number of anomalies detected:", anomaly_count)


# ----- COST COMPONENT BREAKDOWN -----

st.subheader("Cost Component Breakdown")


components = ['compute_cost', 'storage_cost', 'transfer_cost', 'api_cost']

available = [c for c in components if c in df.columns]


if available:

```

```
comp_sum = df[available].sum()

fig4, ax4 = plt.subplots()

comp_sum.plot(kind='pie', autopct='%1.1f%%', ax=ax4)

ax4.set_ylabel("")

st.pyplot(fig4)

else:

    st.warning("Cost component columns not available")

# ----- AUTO INSIGHTS -----

st.subheader("Auto-Generated Insights")

insights = []

highest_service = service_cost.idxmax()

lowest_service = service_cost.idxmin()

insights.append(f"Highest cost is contributed by '{highest_service}'.")

insights.append(f"Lowest cost is from '{lowest_service}'.")

peak_month = monthly_cost.idxmax()
```



```
low_month = monthly_cost.idxmin()
```

```
insights.append(f'Peak cost observed in month {peak_month}.')
```

```
insights.append(f'Lowest cost observed in month {low_month}.')
```

```
if 'seasonal_effect' in df.columns:
```

```
    seasonal_cost = df.groupby('seasonal_effect')['cost'].mean()
```

```
    if True in seasonal_cost and False in seasonal_cost:
```

```
        if seasonal_cost[True] > seasonal_cost[False]:
```

```
            diff = ((seasonal_cost[True] - seasonal_cost[False]) / seasonal_cost[False]) * 100
```

```
            insights.append(f'Seasonal periods show approximately {diff:.1f}% higher cost.')
```

```
if anomaly_count > 0:
```

```
    insights.append(f'{anomaly_count} anomalies detected indicating unusual cost spikes.')
```

```
if available:
```

```
    top_component = comp_sum.idxmax()
```

```
    insights.append(f'{top_component}' is the major contributor to total cost.')
```

```
for ins in insights:
```

```
    st.write("•", ins)
```

```
# ----- SMART RECOMMENDATIONS -----
```

```
st.subheader("Smart Recommendations")
```

```
st.write(f"• Optimize usage of '{highest_service}' to reduce cost.")
```

```
if anomaly_count > 0:
```

```
    st.write("• Investigate anomaly spikes to prevent unexpected billing.")
```

```
if 'seasonal_effect' in df.columns:
```

```
    st.write("• Plan resources in advance for seasonal demand.")
```

```
if available:
```

```
    st.write(f"• Focus on '{top_component}' to achieve maximum cost savings.")
```

```
else:
```

```
    st.warning("Please upload a CSV file to proceed")
```

F.2 Data Analysis Notebook

The notebook was used for initial data exploration, preprocessing, and testing of analysis logic.

Only the key steps are included below for reference.

Dataset Loading

```
import pandas as pd
```

```
df = pd.read_csv("cloud_cost_data.csv")  
df.head()
```

The dataset is loaded using Pandas and a preview is displayed to verify the data.

Data Understanding

```
df.info()  
df.describe()
```

Basic information about the dataset and statistical summary are obtained to understand the structure and distribution of data.

Cost Aggregation

```
service_cost = df.groupby('service_type')['cost'].sum()  
monthly_cost = df.groupby('month')['cost'].sum()
```

The data is grouped to calculate total cost based on service type and month, which helps in analyzing cost distribution and trends.

Anomaly Detection

```
threshold = df['cost'].mean() + 2 * df['cost'].std()  
  
df['Anomaly'] = df['cost'].apply(  
    lambda x: 'Anomaly' if x > threshold else 'Normal'  
)
```

A threshold-based method is used to identify unusual cost values. Data points exceeding the threshold are marked as anomalies.

Sample Visualization

```
import matplotlib.pyplot as plt  
  
monthly_cost.plot(marker='o')  
plt.xlabel("Month")  
plt.ylabel("Total Cost")  
plt.title("Monthly Cost Trend")  
plt.show()
```

A sample visualization is generated to observe how cloud cost varies over time.

Note

The complete Jupyter Notebook used for data analysis is provided separately along with the project submission.